Q26: How Does Fine-Tuning Change a Model — and How Should We Adapt It?

DLS 2026 — Module (Paradise)

October 2026

Learning objectives

After this module, a student should be able to (i) distinguish what fine-tuning changes in weights from what changes on the *behavior surface* (response distribution vs. knowledge), (ii) explain catastrophic forgetting and the main mitigations (replay, low-rank updates), (iii) place LoRA and QLoRA in the parameter-efficient family and say why a rank-r constraint preserves base capabilities, (iv) apply the retrieve-vs-tune decision rule, and (v) reason about online/continual adaptation and why naive online updates destabilize RLHF-trained models.

1 Weight update vs. behavior surface

Fine-tuning is, mechanically, nothing exotic: continue running (a variant of) SGD on a loss computed over a new, narrower dataset, starting from pretrained weights θ_0. What students often miss is that the *mechanics* say nothing about which *kind* of change the user will observe. Two families dominate practice:

- **Behavior reshaping.** Instruction tuning and preference optimization (RLHF, DPO) change the *response distribution* — how the model formats, refuses, plans, and engages — far more than what it knows. InstructGPT is the canonical demonstration: a 1.3B model tuned on demonstrations and preference labels was *preferred* by human raters to the 175B base GPT-3 it came from, on the same prompt distribution [6]. The knowledge was already in the base model; the tuning moved probability mass from "continue this text" toward "answer this request." DPO makes the same point from the optimization side: preference alignment is a classification-like loss on the policy itself, with a KL anchor to the reference model to keep the change bounded [8].
- **Knowledge/vocabulary injection.** Continued (domain) pre-training on in-domain corpora is meant to add vocabulary, entities, and facts. This is the harder direction: the evidence is that unsupervised fine-tuning transfers new *facts* poorly. Ovadia et al. compared unsupervised fine-tuning against retrieval-augmented generation (RAG) on knowledge-intensive tasks and found RAG consistently ahead — for knowledge the model had seen and for genuinely new knowledge alike; fine-tuning helped somewhat but "was not competitive" [7].

This split motivates the module's decision rule: *if it's facts you need, retrieve; if it's behavior you need, tune.* Facts change hourly, cite poorly from weights, and audit worse; retrieval (RAG couples a frozen generator to a retrievable non-parametric memory [5]) updates by editing an index, with provenance for free. Behavior — tone, format, tool protocol, refusal style — cannot be prompted reliably at the margin and is exactly what gradient descent on demonstrations moves cheaply.

2 Catastrophic forgetting

The obstacle to any sequential adaptation is *catastrophic forgetting*: training on task B overwrites weights that task A relied on, because plain SGD has no memory of which weights matter to what [2, 4]. The effect is abrupt, not gradual — classic list-learning results show accuracy on old associations collapsing within a handful of new-task trials [2]. Two mitigations matter for LLM practice:

- **Replay (data mixing).** Interleave a sample of pre-training or general instruction data with the domain data. The gradient signal on old-task data cancels the drift; this is why every serious domain-adaptation recipe mixes replay buffers, and why continual pre-training uses much lower learning rates [7].
- **Constrained updates (LoRA, EWC-style penalties).** Elastic Weight Consolidation penalizes movement of weights proportionally to their importance (a Fisher-information diagonal) to old tasks [4]. LoRA reaches a similar endpoint by construction rather than penalty, as the next section explains.

3 The PEFT family: LoRA and QLoRA

LoRA [3] freezes the pretrained weight matrix $W_0 \in \mathbb{R}^{d\times k}$ and learns only a low-rank residual:

$$W = W_0 + \Delta W, \qquad \Delta W = \frac{\alpha}{r} BA, \qquad B \in \mathbb{R}^{d\times r},\ A \in \mathbb{R}^{r\times k},\ r \ll \min(d,k). \tag{1}$$

Only A and B receive gradients (A initialized randomly, B at zero, so $\Delta W = 0$ at step 0). Three consequences:

1. **Parameter count.** Trainable parameters drop from dk to $r(d+k)$; on GPT-3 175B, up to 10,000× fewer, with no added inference latency because $W_0 + BA$ can be merged into one matrix [3].
2. **Why rank-r preserves base capabilities.** The update lives, by construction, in an r-dimensional subspace of a d-dimensional row space: ΔW's rows are all linear combinations of the r rows of A. The bulk of W_0 — the directions that encode pretrained competence — is untouched, and the residual the optimizer may express is confined to a subspace too small to rewrite the base function wholesale. Hu et al.'s empirical finding that the adaptation-induced ΔW has low "intrinsic rank" (very small r suffices even for $d = 12{,}288$) is what makes this a *constraint* that costs little rather than a straitjacket [3]. The forgetting story is a corollary: a capacity-limited residual cannot drift as far from W_0 as an unconstrained full update, which is why LoRA behaves like a soft EWC in our companion demo.
3. **Swappability.** Adapters are small enough to store per domain and hot-swap at inference; the base model serves all domains.

QLoRA [1] stacks a quantization trick on top: backpropagate through a frozen *4-bit* quantized base model (NF4, an information-theoretically motivated datatype for normally distributed weights, plus double quantization of the constants and paged optimizers) into LoRA adapters. The result — a 65B model fine-tuned within a single 48GB GPU with 16-bit-level task performance — moved fine-tuning from a datacenter activity to a workstation one, at some cost in fidelity and throughput relative to full precision.

4 RAG vs. fine-tune: the decision rule in practice

The rule is a first cut, not a law. Retrieve when: knowledge is volatile, must be cited, is too large to encode, or access must be revocable. Tune when: the target is *behavior* — output format, style, domain-specific instruction-following, latency-critical skills that prompting cannot stabilize — or when the domain's *style* (not its facts) is the shift. Combine them when both apply: a LoRA-tuned generator over a domain index is a standard stack, and Ovadia et al. found fine-tuning the RAG generator sometimes helps, though inconsistently — evidence that fine-tuning's effect on knowledge injection is itself unstable [7]. Costs favor retrieval for iteration speed (an index rebuild is not a training run), and tuning for per-token inference cost and no context-window tax.

5 Online and continual adaptation

Test-time training. Adaptation can happen at inference: Sun et al.'s test-time training turns each unlabeled test input into a self-supervised problem and updates (a copy of) the model before predicting, extending naturally to streams [9]. In LLM use, the same idea appears as test-time/few-shot adaptation and cache/context editing; the discipline it teaches is that the objective used online must be *self-supervised* — there are no labels at test time.

Streaming updates. Full continual learning — weights updated as data arrives — remains fragile. Non-stationary streams, small per-step batches, and distribution drift reintroduce catastrophic forgetting at production cadence; replay ratios, EWC-style consolidation [4], and bounded updates are the standard controls.

Why naive online SGD destabilizes RLHF models. A preference-tuned model sits at a delicate optimum: a KL-anchored point where small logit shifts can move generation out of the trained distribution. Unconstrained online updates (i) violate the anchor — there is no reference-model penalty in plain SGD, so the policy drifts arbitrarily far from what the preference data certified; (ii) sharpen the distribution — repeated likelihood-style updates on a narrow stream collapse entropy, producing repetitive, over-confident outputs; and (iii) compound — each update moves the starting point of the next, so errors accumulate. This is why deployed "learning" systems favor bounded mechanisms (LoRA residual with small r, KL/anchor penalties, periodic re-tuning from the frozen base [8]) over in-the-loop weight writes.

6 Adaptation pipelines in practice

A domain-adaptation stack, in the order that works:

1. **Retrieval first.** Build the domain index; measure how far the frozen base + RAG already goes. This is the cheapest iteration loop [5, 7].
2. **SFT for domain behavior.** If format/instruction-following still lags, SFT on curated domain demonstrations — typically LoRA [3] or QLoRA [1] — with replay data mixed in.
3. **Preference optimization last.** DPO/RLHF stages shape tone and refusals; run them *after* SFT, because preference data assumes the model can already produce the behavior being ranked, and the KL/DPO anchor is defined against the post-SFT reference [8, 6]. Reversing the order wastes the preference stage re-teaching format.

The one-sentence summary: *fine-tuning is a constrained, forget-prone edit to a response distribution — retrieve for knowledge, tune for behavior, keep the update bounded, and order the pipeline SFT → preference.*

References

[1] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. Qlora: Efficient finetuning of quantized llms. In *Advances in Neural Information Processing Systems*, 2023.

[2] Robert M. French. Catastrophic forgetting in connectionist networks. *Trends in Cognitive Sciences*, 3(4):128–135, 1999.

[3] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022.

[4] James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz, Joel Veness, Guillaume Desjardins, Andrei A. Rusu, Kieran Milan, John Quan, Tiago Ramalho, Agnieszka Grabska-Barwinska, Demis Hassabis, Claudia Clopath, Dharshan Kumaran, and Raia Hadsell. Overcoming catastrophic forgetting in neural networks. *Proceedings of the National Academy of Sciences*, 114(13):3521–3526, 2017.

[5] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive nlp tasks. In *Advances in Neural Information Processing Systems*, 2020.

[6] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul F. Christiano, Jan Leike, and Ryan Lowe. Training language models to follow instructions with human feedback. *Advances in Neural Information Processing Systems*, 2022.

[7] Oded Ovadia, Menachem Brief, Moshik Mishaeli, and Oren Elisha. Fine-tuning or retrieval? Comparing knowledge injection in LLMs. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 237–250, 2024.

[8] Rafael Rafailov, Archit Sharma, Eric Mitchell, Christopher D. Manning, Stefano Ermon, Chelsea Finn, and Sergey Levine. Direct preference optimization: Your language model is secretly a reward model. In *Advances in Neural Information Processing Systems*, 2023.

[9] Yu Sun, Xiaolong Wang, Zhuang Liu, John Miller, Alexei A. Efros, and Moritz Hardt. Test-time training with self-supervision for generalization under distribution shifts. In *International Conference on Machine Learning*, 2020.